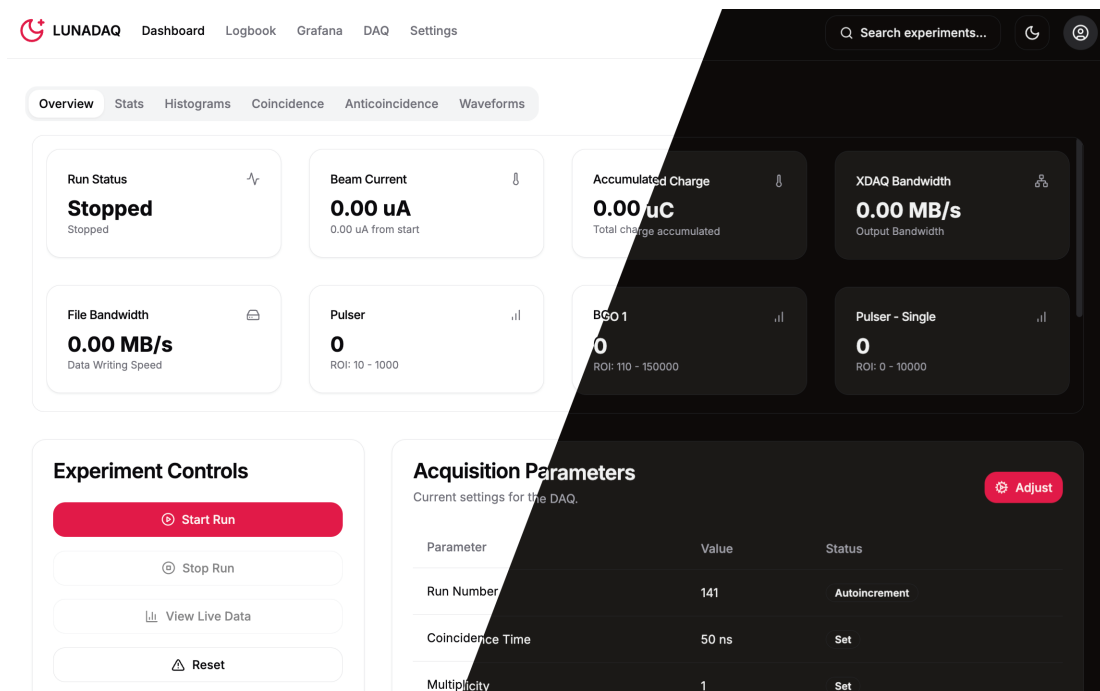


WebDAQ

Data acquisition for the LUNA experiment

Operator and technical manual

Version 5.0



A. Compagnucci, R. Gesuè, J. Skowroński

INFN Laboratori Nazionali del Gran Sasso and Università di Padova

2nd October 2026

Contents

1	What WebDAQ is	3
1.1	The data path	3
1.2	The three pieces	3
1.3	The working directory is the experiment	3
2	Installing and starting	4
2.1	What the machine needs	4
2.2	Installing	4
2.3	Starting it	4
2.4	Running it in tmux	5
2.5	Without hardware	5
3	Taking a run	6
3.1	Before the beam	6
3.2	Starting	6
3.3	While it runs	7
3.4	Stopping	7
3.5	Afterwards	7
4	The screens	8
4.1	Dashboard	8
4.1.1	Overview	8
4.1.2	Board Health	8
4.1.3	Data Rates	9
4.1.4	Histograms, Waveforms and PSD	9
4.2	Logbook	9
4.3	Stats	9
4.4	DAQ	9
4.5	Tuner	10
4.6	Troubleshoot	10
4.7	Alerts	10
4.8	Settings	10
5	Boards and synchronisation	11
5.1	Adding a board	11
5.2	What a synchronised run is	11
5.3	What each position needs	11
5.4	Clock as well as start	12
6	Spectra and regions of interest	13
6.1	Regions of interest	13
6.2	An empty spectrum	13

7	Beam current and charge	14
7.1	Two charges, and why they differ	14
7.2	Three sources	14
7.3	Freshness	14
8	Watching the experiment	15
8.1	Alerts are rules	15
8.2	Why the messages are worth reading	15
8.3	Auto-restart	16
8.4	What happened while nobody was looking	16
9	Run data and conversion	17
9.1	What a run leaves behind	17
9.2	Converting	17
9.3	Checking an interrupted run	17
10	When something is wrong	19
10.1	The five recovery actions	19
10.2	Symptoms	19
10.3	After a crash or a power cut	20
10.4	The log	20
A	Configuration files	21
B	Environment variables	22
C	Running the tests	23

Chapter 1

What WebDAQ is

WebDAQ reads the CAEN digitizers of the LUNA setup, writes the data, fills the spectra while they fill, records what the beam was doing, and says something when a board stops. It is used from a browser, by the person on shift, and it is meant to be learnt in an evening.

1.1 The data path

digitizers → CaenDAQ, inside the server process
→ `run_<N>_0000.caendat` on disk
→ spectra, rates, regions of interest
→ Graphite, for the slow-control archive

Acquisition runs *inside* the DAQ server. The boards are opened, read, decoded and written by CaenDAQ, a C++ library bound into Python, so pressing **Start** is the whole operation: there is no container to launch, no state machine to drive and no second process to keep alive. The spectra you watch are the histograms CaenDAQ is already accumulating while it decodes; the server takes a snapshot of them when the page asks.

Versions before 4.0 ran acquisition as XDAQ inside a Docker container, with a separate spy server on port 6060. Any procedure that mentions a container, port 6060 or a `topology.xml` predates 4.0 and does not apply.

1.2 The three pieces

- **The DAQ server** — Flask and waitress on port 5001. It owns the boards, the run, the files and the configuration.
- **The web interface** — Next.js on port 3000. It holds no state of its own worth protecting; everything it shows comes from the server.
- **RUReader** — the offline converter from `.caendat` to ROOT, run on request from the Logbook.

1.3 The working directory is the experiment

Everything the server reads and writes is relative to the directory it was started in: `conf/` (board configurations, the histogram dashboard, credentials), `calib/`, `data/` and the `app.db` database. Two working directories are two independent experiments that happen to share one installation.

This is worth knowing when something looks *missing*. A dashboard with no histograms, or an empty board list, usually means the server was started somewhere other than where the campaign was configured. That is a different setup, not a broken one, and the **DAQ Server** panel names the directory it is running in.

Chapter 2

Installing and starting

2.1 What the machine needs

Linux for a DAQ machine, macOS for development. For real hardware, the CAEN libraries (CAENVMelib, CAENComm, CAENDigitizer); without them WebDAQ still runs in test mode with simulated boards. Everything else — Python, ROOT, the acquisition module — lives in a conda environment called `luna`, created by the installer.

2.2 Installing

```
git clone https://github.com/skowrons94/WebDAQ.git
cd WebDAQ
./install.sh
```

The installer is idempotent: every step checks whether the work is already done, so running it again after an update is safe. It installs the build tools, checks out the `CaenDAQ` and `RUReader` submodules, installs `Miniforge` if `conda` is missing, creates the `luna` environment, builds `RUReader` and the `caendaq` module, writes `frontend/.env`, builds the interface, and adds a `LunaDAQ` launcher to `~/.bashrc`.

The API address is compiled in. `frontend/.env` holds `NEXT_PUBLIC_API_URL`, and `Next.js` bakes it into the bundle at build time. If the server is to be reached from other machines, set it to an address those machines can resolve — not `127.0.0.1` — and rebuild with `npm run build` in `frontend/`. Forgetting the rebuild is the most common deployment mistake: the interface loads, then fails every request.

2.3 Starting it

```
LunaDAQ          # start the web interface (freeing its port first)
LunaDAQ status    # what is holding each port
LunaDAQ stop      # stop both parts, politely then firmly
LunaDAQ restart   # stop, then start the interface
LunaDAQ backend   # run the DAQ server in this terminal, for debugging
```

Open `http://localhost:3000` and log in. The **DAQ Server** badge at the top right starts the server itself: choose the working directory for the campaign and press **Start**. The badge is also on the login page, so the server can be started before anyone logs in.

`LunaDAQ stop` only stops processes that look like ours. An unrelated program on port 3000 is reported and left alone, which is why it is a better answer to a busy port than killing whatever holds it.

2.4 Running it in tmux

The web interface is a long-lived process: started from a plain SSH session it dies when the connection drops. Start it inside **tmux**, a session that keeps running on the machine whether or not anyone is attached to it.

```
tmux new -s daq      # create a session called "daq" and attach to it
LunaDAQ              # start the interface inside it
```

Then detach and leave it running: press **Ctrl-b**, release both keys, and press **d**. You are back in your own shell and the session is still running.

What you want	How
List the sessions on the machine	<code>tmux ls</code>
Attach again	<code>tmux attach -t daq</code>
Detach, leaving it running	Ctrl-b then d
Take over a session someone left attached	<code>tmux attach -d -t daq</code>
Scroll back through the output	Ctrl-b then <code>[</code> , arrows or PageUp; q to leave
A second window in the session	Ctrl-b then c ; switch with Ctrl-b then 0 , 1 , ...
Close a window	type <code>exit</code> in it
Stop everything	<code>attach, LunaDAQ stop</code> , then <code>exit</code>

Every tmux command is the prefix **Ctrl-b** followed by one key; that is the only thing worth memorising. A detached session survives a dropped SSH connection, but not a reboot of the DAQ machine — after a reboot, start a new session.

2.5 Without hardware

```
TEST_FLAG=True LunaDAQ backend
```

or tick **Test Mode** in the **DAQ Server** panel before starting. Test mode fabricates CAEN-style data for simulated boards and simulates the picoammeter, so the whole chain works on a laptop. Set the variable or leave it unset; **never set it to False** — most of the code reads any value at all as “test mode”.

Chapter 3

Taking a run

3.1 Before the beam

1. **Settings** → **Boards**: every board *Connected*. Use **Scan for boards** rather than typing settings from memory; the scan fills the add form and leaves the ID and the firmware choice to you.
2. **DAQ** → **Configuration**: thresholds and shaping for the measurement. For several boards, check **Synchronization** (chapter 5).
3. **Settings** → **Current Module**: where the beam current comes from — a TetrAMM, an RBD 9103, or a value already monitored in Graphite.
4. **Stats**: the accelerator values this campaign should record. Their name and unit become the column headings of every run’s `stats.csv`.
5. **Settings** → **Notifications**: at least one destination and the alerts worth having (chapter 8).

3.2 Starting

On **Dashboard** → **Overview**, in **Run Setup & Control**, fill in **Run Name**, **Run Type**, **TV (kV)** and **PV (kV)** — all four are pre-filled from the last run — and press **Start Run**.

What happens, in order: the run number and the saving options are stored; the beam-current recording and the statistics are pointed at this run; the boards are handed to CaenDAQ, configured and started; the run state is set, which starts the charge integration and the board monitor; and the run’s metadata row is written with the hardware and software provenance read back from the open boards.

Two refusals are worth knowing in advance. A run will not start without a run number, and a run directory that already holds data is never silently reused — you are asked first.

A run also starts when something *next to* it fails. If the current module or the statistics cannot be started, the run starts anyway and the message names what is missing: “*Run 1276 is acquiring, but beam current is not being recorded*”. A run without its current log is worth more than no run.

3.3 While it runs

Where	What it answers
Overview, board cards	Is every board still <i>Running</i> ?
Board Health	What has each board actually read, written and lost?
Data Rates	Counts per second, per channel, for one board.
Histograms	Are the spectra filling where they should?
Beam Control	Current on target, charge this run, lifetime charge.
Header	Run state, elapsed time, and the alert bell.

Changing settings mid-run is covered in section 4.5.

3.4 Stopping

Stop Run stops the monitor, the spy snapshots and the acquisition, hands the boards back, writes the end time, advances the run number (when saving) and sets the run state to stopped — which stores the accumulated charge, closes `current.txt` and writes `roi.json`. Auto-managed Grafana rules are silenced again.

3.5 Afterwards

Open the run in the **Logbook**: notes in Markdown, the beam current it was taken with, the monitored metrics beside it, the register dump of every board, and conversion to ROOT. **Logbook** → **ELOG** → **New entry** → **Write up a run** drafts a logbook entry from the run's own record.

Chapter 4

The screens

The navigation is the same everywhere: **Dashboard**, **Logbook**, **Stats**, **DAQ**, **Tuner**, **Troubleshoot**, **Alerts**, **Settings**. The header carries the run state with an elapsed clock, the **DAQ Server** badge, the alert bell, and a **Search** palette that jumps to any page or tab.

Logging in is per browser and the session does not expire on its own. If it is cleared, the notice says plainly that the run is unaffected: the DAQ server keeps taking data with nobody logged in.

4.1 Dashboard

Six tabs: **Overview**, **Board Health**, **Data Rates**, **Histograms**, **Waveforms**, **PSD**. The last three can be switched off in **Settings** → **Appearance**.

4.1.1 Overview

System status is the band of cards at the top; drag the handle beneath it to show one row more or less.

Card	What it tells you
Run Status	Run number, running or stopped, how long ago it started, the file write rate, and a Save switch. While stopped, the run number is a button that opens <i>Next run</i> .
DAQ Status	Data saving, waveforms, current, stats — each with a tick or a cross, so you can see what this run will actually record.
Picoammeter	For a real device only: connected or not, whether it is sampling, its address and range. A monitored Graphite value has no device card.
Board n	<i>Ready</i> , <i>Running</i> , <i>Board Failure</i> or <i>Disconnected</i> , with a waveform switch that is locked during a run.
ROIs, Metrics	Live region-of-interest integrals, and the Graphite values selected on the Stats page.

Acquisition setup shows what this run will do — saving, the file size limit, the auto-restart delay — and **Adjust** opens the dialog that sets them. **Beam Control** plots the current with the charge beside it and can be moved to one of four places on the page.

4.1.2 Board Health

One card per board with the readout counters CaenDAQ keeps for the run: data blocks read, events decoded, bytes read and bytes sent to file, each with its rate, and then the three numbers that matter when something is wrong.

Counter	Read it as
FAIL blocks	The board could not sustain the readout for that block — a full buffer or a lost link. Data from that point may be incomplete.
Dropped	Blocks the write queue refused because it was full. This is lost data and should always be zero.
Read errors	CAEN read errors on the link.

The badge is the quick answer: *Reading, No data for n s, FAIL flag, No answer* (the last request failed, so the numbers are the last known ones) or *Not reported* (this CaenDAQ build does not publish that counter). Every counter belongs to the run: they start at zero and are gone when it ends, which is why the tab says so between runs rather than showing zeros.

4.1.3 Data Rates

A tile per board, four tiles for the selected board — events, pile-ups, lost events, saturations — and the write rate. Pile-up, lost and saturation turn amber when they are not zero.

The **Plot** control sets the sampling cadence, and it is not a display setting: one CaenDAQ tick samples the counters, differences them and publishes them, so the same number sets the refresh rate, the averaging window and the resolution of what reaches Graphite. **Rate shown** switches between the four rates without losing the trace, because all four are recorded.

4.1.4 Histograms, Waveforms and PSD

The histogram dashboard belongs to the experiment, not to your browser: the layout, the regions of interest, the zoom and the display settings live on the DAQ server in `conf/histograms.json`, so every screen in the control room shows the same thing and a restart changes nothing. Chapter 6 covers it.

4.2 Logbook

Two logbooks: the run metadata WebDAQ records itself, and the collaboration’s ELOG. The run table is searchable and filterable, the flag and the notes are editable in place, and **Export CSV** writes out exactly the rows you filtered to. A run number opens the run’s own page with six tabs: **Overview**, **Notes**, **Beam Current**, **Stats**, **Boards** and **Convert** (chapter 9).

4.3 Stats

The Graphite values this experiment watches, each with its latest reading, a sparkline and a timestamp. The light at the top says whether Graphite is answering at all, so a dead server never looks like a metric with no data. **Add metric** browses or searches the metric tree. The **name and unit** you give a metric become the column heading in every run’s `stats.csv`.

The **Graphite server** card holds the host, the port of the render API, and the **metric prefix** for this experiment. Give each campaign its own subtree — `ancillary.rates.12c12c` — and no two campaigns share a series.

4.4 DAQ

Configuration is the full register set of one board, grouped by function, in the order the signal is processed; **Advanced** shows every register and **Binary** shows bit fields. **Writes on this tab reach the board immediately and are not blocked during a run** — it is the same

dashboard used to set a board up, so it is not the place to experiment mid-measurement. Use the Tuner, which knows what is safe.

Synchronization is the daisy chain (chapter 5). **Calibration** holds two numbers per channel for the energy axis. **Hardware Info** is what the boards report about themselves — model, serial number, firmware, licence, and the acquisition registers as programmed. The CAEN API answers only while the digitizers are open, so that page is populated during a run.

4.5 Tuner

The place to change thresholds and shaping with beam on target. It starts acquisition with **data saving switched off**, so nothing lands in a run directory, and a badge reminds you that the Tuner is holding the run.

The **Online** switch decides where a change goes. With it on during a run, only registers on the firmware’s safe list are written to the board; every field is marked *live* or *next run before* you touch it. The save to the configuration always happens, and a write the board refuses is reported as *saved, not sent to the board* with the reason — so the configuration and the hardware cannot drift apart.

4.6 Troubleshoot

Recovery at the top, **History** below. Chapter 10.

4.7 Alerts

The Grafana alert rules, managed without leaving run control. Marking a rule **auto-managed** makes it active while a run is going and silent between runs, which is the answer to a “beam current too low” rule that is right during a run and pure noise while you are setting up.

4.8 Settings

Six sections: **Boards**, **Current Module**, **Notifications**, **ELOG**, **Grafana** and **Appearance**. Everything except Appearance is stored on the DAQ server, so it is the same for everyone.

Credentials. `conf/telegram_settings.json`, `conf/zulip_settings.json`, `conf/elog_settings.json` and `conf/grafana_settings.json` hold secrets in plain text on the DAQ machine. The interface masks them once saved and never shows them again, but the files are readable by anyone with an account on that machine — so do not copy a working directory between sites without emptying them.

Chapter 5

Boards and synchronisation

5.1 Adding a board

Settings → **Boards** → **Scan for boards** probes the links — USB, optical (CONET), A4818, and a VME crate through a bridge — and lists the digitizers that answer with their model, serial number, channel count and firmware. **Use these settings** fills the add form rather than adding the board, because the board ID and the DPP firmware are judgements, not measurements: the firmware is only *inferred* from the code the board reports.

Adding a board reads its register dump out of the hardware into `conf/<model>_<id>.json` and creates a unit calibration in `calib/`. That JSON file is the source of truth from then on: the Tuner, the CAEN dashboard, the waveform switch and CaenDAQ itself all read it, and it is copied into every run directory.

5.2 What a synchronised run is

Synchronisation is never set by WebDAQ. It comes from the boards' own registers, and the server reads it back rather than remembering what it was told. A board whose start mode (Acquisition Control 0x8100, bits [1:0]) is anything other than *software* is *armed* rather than started, and waits.

CaenDAQ then starts the run like this: align the clock chain, arm every synchronised board, and only then fire one software trigger on the **master** — the synchronised board whose register id is 0. That pulse starts the master and leaves its TRG-OUT, enters the next board's TRG-IN, and walks the chain, so every board shares one time origin.

5.3 What each position needs

Board	Needs
The master	<i>On first trigger</i> mode, and its software trigger routed to TRG-OUT (0x8110 bit 31) when any board follows it. It does not need TRG-IN → TRG-OUT: no trigger arrives on its TRG-IN, it makes its own.
A board in the middle	TRG-IN → TRG-OUT (0x8110 bit 30), so the pulse walks on, and TRG-OUT carrying the trigger rather than a probe (0x811C bits [17:16] = 0).
The last board	Nothing on its TRG-OUT. The chain ends there.

DAQ → **Configuration** → **Synchronization** checks all of this for you and marks each switch *needed* — *on*, *needed* — *off* or *not needed here* for that board's position. Anything that would stop the chain starting is listed against the board that causes it, in words rather than

register numbers. Getting this wrong is expensive and silent — the run starts, one board never sees a trigger, and its data is short — which is why the check exists.

The master is identified from the register ids the hardware reports. Those are readable only while the boards are open, so between runs the panel says the master is *assumed* from the configuration.

5.4 Clock as well as start

Synchronising the *start* aligns the boards at $t = 0$; sharing a *clock* keeps them aligned. On their own oscillators the boards drift apart over a long run. Leave the first board on *Internal*, feed its CLK-OUT to the next board's CLK-IN, and set every downstream board to *External CLK-IN* (0x8100 bit [6]; desktop and NIM digitizers only — in a VME crate the clock comes from the crate). CaenDAQ re-aligns the chain at every run start and reports a board whose PLL did not stay locked, rather than refusing to run.

Chapter 6

Spectra and regions of interest

The spectra are the ones CaenDAQ accumulates while decoding; the server snapshots them when a page asks. Nothing is transported twice and there is no monitoring process to fall behind.

The dashboard belongs to the experiment. Which spectra are shown, their labels, their regions of interest, their zoom and the display settings live in `conf/histograms.json` on the DAQ server. A second screen in the control room shows the same dashboard, and a restart changes nothing.

6.1 Regions of interest

A region is a name, a range and a colour, defined per spectrum. Integrals are shown live on the Overview and on the spectrum itself. At every run stop, every defined region — visible or not, enabled or not — is written into the run directory as `roi.json`, with its bounds and its counts, so a run carries the regions it was analysed with.

6.2 An empty spectrum

An empty histogram says why in its own title: *No data yet — start a run*, *This board is not part of the current run*, *No waveform — switch waveforms on for this board*, or *No counts yet on channel n* . Read the title first; then check on **Board Health** whether the board is reading data blocks at all, and only then the channel enable and the threshold.

Chapter 7

Beam current and charge

7.1 Two charges, and why they differ

This run integrates for exactly the length of the run and is stored in the run’s metadata. **Lifetime total** never stops, so it can serve as a target history. A figure that does not move between runs is correct behaviour.

The run figure follows the DAQ’s own run state rather than the browser: a closed tab, an auto-restart or a run that saves nothing all leave it correct. A dot next to *This run* says whether it is integrating.

7.2 Three sources

Module	What it is
TetrAMM	Four-channel picoammeter over TCP. One channel is the charge channel.
RBD 9103	Single-channel picoammeter over a serial port.
Monitored value	No hardware: the beam current is read from a value already published to Graphite — the accelerator’s own readout — scaled into microamps. It plots, logs and integrates exactly like a picoammeter.

7.3 Freshness

A picoammeter answers in milliseconds, so a reading older than 30 s means it has stopped. A monitored value is different: it is published on its own cadence, Carbon flushes on its own schedule, and the render API will not serve the bucket it is still filling — so a value written every 10 s legitimately reads back tens of seconds old. WebDAQ measures the cadence and the lag it actually sees and derives the freshness horizon from them, which is why a healthy slow metric is no longer reported as a dead readout.

Non-finite and negative readings are refused rather than integrated, and the length of gap that may be integrated across is bounded — so a Graphite outage cannot add charge that was never delivered.

Chapter 8

Watching the experiment

Four different things watch a run, and they are easy to confuse.

Tool	What it does here
Graphite	Stores every number anyone measures: beam current, terminal voltage, board rates. WebDAQ writes to it and reads from it.
Grafana	Draws those numbers on the control-room screen and raises its own alerts.
The Stats page	Shows the current value of the metrics <i>you</i> selected, and copies them into each run's <code>stats.csv</code> .
Board Health	The readout counters of each board, live for the run only.

8.1 Alerts are rules

An alert is a rule: what to watch, the threshold, how long the condition has to hold, and which destinations carry it. Rules live in `conf/alerts.json` and are edited in **Settings** → **Notifications**.

Alert	Fires when
Board failure	A board sets its FAIL flag during a run.
Board stopped producing data	A board reads no new data block for n seconds during a run, whether or not it raised the FAIL flag.
Beam current	The current stays below (or above) a threshold for n seconds.
Graphite metric	A monitored value stays outside its range. One rule per metric.

Destinations are **Telegram** (a bot posting to a chat or group) and **Zulip** (a bot posting to one stream and topic). Each rule names its own, so a board failure can wake the whole shift while a current dip reaches one group.

8.2 Why the messages are worth reading

- **One message per episode.** A rule fires when its condition starts and stays quiet until it clears, then says it cleared.
- **Unknown is not wrong.** An unreachable Graphite, or a monitor that stopped, leaves every rule exactly as it was — a hole in the monitoring never invents an alert and never silently clears a real one. Such a rule is marked *nothing to measure*, so it is not mistaken for a happy one.

- **A flap is one message.** The same subject is not announced twice within a minute.
- **An alert that reached nobody says so.** The bell turns amber and the history row is badged *reached nobody*.

A rule only works while the server is *watching*; the panel says so and offers **Start watching** when it is not. A board failure is recorded in the history **even when no rule is configured for it**.

8.3 Auto-restart

With auto-restart on, a board failure stops the run, flags it *bad* with a note, and starts a new one after the delay. The new run continues the failed one: its own `current.txt`, charge and `stats.csv`, and the target, voltages and run type carried over. The intent is that a night shift loses one run rather than the rest of the night.

8.4 What happened while nobody was looking

Every alert and every recovery is written on the server (`conf/alert_log.json`, the most recent 500 events, kept across restarts). The **bell** carries the unread count; **Troubleshoot** is the full history, with the time, what it was about, which run it belonged to, and which destinations received it. This is the page to open after a night shift.

Chapter 9

Run data and conversion

9.1 What a run leaves behind

Everything goes into `data/run<N>/`, and only when saving is on.

File	What it is
<code>run_<N>_0000.caendat</code>	The data: one unified stream for every board, with the same header layout the XDAQ-era files had. Further parts appear when a size limit is set.
<code><model>_<id>.json</code>	The board's complete register dump, copied from <code>conf/</code> at run start. This is what makes the run reproducible.
<code>current.txt</code>	The beam current, one row per sample, time relative to the start of the log.
<code>stats.csv</code>	The monitored accelerator values, one row per sampling tick, with the aliases and units you chose.
<code>stats.csv.part1, ...</code>	Earlier rows of the same run, kept aside when the statistics were restarted mid-run. The run report reads them together with <code>stats.csv</code> .
<code>metadata.json</code>	The run's record, rewritten after every change, so a copied directory is self-describing.
<code>roi.json</code>	Every region of interest with its counts, written at stop.
<code>run<N>.root</code>	The converted file, on request.

The `.caendat` file has no trailer. That is deliberate: a run killed by a power cut is still readable up to the point where it stopped.

9.2 Converting

Logbook → **the run** → **Convert** runs `RUReader` over every `.caendat` in the directory, in name order so a run split across files keeps its time order, and writes `run<N>.root` next to them. The options offered are read from the installed converter rather than assumed, so a build that does not support a flag does not show it. Conversion runs on the server: you can leave the page.

9.3 Checking an interrupted run

```
cd server
python scripts/check_run_integrity.py data/run1276
```

This walks the file structure without converting anything and reports whether the header, the board definitions and the aggregate chain are consistent — the first thing to do after a crash, a full disk or a power cut.

Chapter 10

When something is wrong

Start at the Troubleshoot page. It lists the state of each part of the system, offers the action that repairs it, and keeps the history of what has already happened.

10.1 The five recovery actions

Action	Use it when	During a run
Reopen the boards	A board reads <i>Disconnected</i> after a link glitch.	No
Reset the acquisition	A run failed or the boards are wedged.	No
Reconnect the current monitor	The current reads disconnected or stops updating.	Yes
Restart the statistics	<code>stats.csv</code> stopped being written mid-run.	Yes
Re-check Graphite	After a Graphite outage.	Yes

Nothing here runs by itself. The two that would cost data during a run are refused with the reason written under the button. Restarting the statistics never overwrites measurements: the rows already recorded are moved to `stats.csv.part1` first, and if they cannot be moved the action refuses rather than truncating the file.

10.2 Symptoms

A board reads *Disconnected*

Nothing opened it, or something else holds it. *Reopen the boards* with the run stopped; check that no second backend is running (LunaDAQ `status`); check the cable and the link number.

A board shows *Board Failure*

It set its FAIL flag: it could not sustain the readout, and data from that point may be incomplete. Read the three problem counters on **Board Health** — dropped blocks point at the disk or the write rate, read errors at the link. The run should usually be restarted.

A board stops producing data without failing

Board Health shows *No data for n s*. The FAIL flag was not raised, so suspect the trigger, the cabling into TRG-IN, or a chain that never started.

A synchronised run never starts

DAQ → **Synchronization** names the board at fault and what it is missing. See chapter 5.

Every metric reads *N/A*

Graphite is not answering — the light says which case it is. *Re-check Graphite* also clears the circuit breaker an outage leaves behind.

An alert never arrived

Check, in this order: is the panel *Watching*? Is the rule badged *reaches nobody* (no destination

on) or *nothing to measure* (nothing to read)? Is the bell amber? The history records a board failure even with no rule configured.

Something looks “missing”

Almost always the working directory: the server was started somewhere else, so it is showing a different configuration, not a broken one.

10.3 After a crash or a power cut

Start the server again — a `running: true` left behind is cleared with a warning, because acquisition cannot survive a restart. Check the last run directory with `check_run_integrity.py`, convert it (the option that skips a truncated final aggregate is passed automatically when the installed converter supports it), and read the Troubleshoot history for what the DAQ saw before it stopped.

10.4 The log

`server.log`, in the working directory the server was started in. **DAQ Server** → **Show logs** shows the last 200 kB of it in the browser.

Appendix A

Configuration files

All paths are relative to the working directory.

File	Holds
conf/settings.json	Run number, data saving, size limit, the board list, auto-restart and its delay. Written atomically under a lock; an unreadable one is kept rather than replaced.
conf/<model>_<id>.json	A board's full register dump.
conf/histograms.json	The histogram dashboard, its ROIs and display settings.
conf/stats.json	The Graphite render API, the metric prefix, the sampling cadence and the selected metrics.
conf/current.json	Which current module is active and its settings, plus the Carbon ingest address and the lifetime charge.
conf/alerts.json	The alert rules.
conf/alert_log.json	The alert history, most recent 500 events.
conf/telegram_settings.json	Bot token and chat id.
conf/zulip_settings.json	Site, bot account, stream and topic.
conf/elog_settings.json	Logbook URL, shared account, default attributes.
conf/grafana_settings.json	Grafana address and service-account token.
calib/<model>_<id>.cal	Two calibration numbers per channel.
app.db	Users and run metadata.
server.log	The server's log.

Appendix B

Environment variables

Variable	Effect
TEST_FLAG	Simulated boards and a simulated picoammeter. Set it or leave it unset; never set it to False .
DATABASE_URL	Where the database lives. Defaults to <code>app.db</code> in the working directory.
SECRET_KEY, JWT_SECRET_KEY	Flask and token signing keys. Both have insecure defaults; set them on a machine that is reachable from outside.
NEXT_PUBLIC_API_URL	The API address compiled into the interface.
WEBDAQ_SKIP_DB_UPGRADE	Skip the schema check at startup.
WEBDAQ_FORCE_RESTART	Start even over a backend that is taking data.
RUREADER_BIN	Path to the converter, when it is not on <code>PATH</code> .

Appendix C

Running the tests

```
cd server
conda activate luna
./tests/run_tests.sh           # everything
./tests/run_tests.sh alerting  # only modules matching test_*alerting*.py
```

The suite runs with simulated boards in a scratch working directory, so it never touches a real `conf/`, `data/` or database. Tests that drive the HTTP surface read the directory they are started from, which is why the script creates one.